When Telemetry-Driven Interventions Don't Transfer: A Cross-Tier Replication Study of Closed-Loop Agent Recovery via Vendor Agent CLIs for Edge-Cloud Deployments

Krishna Chaitanya Balusu
Independent Researcher
krishnabkc15@gmail.com

Three Actionable Insights for Practitioners

- **Validate published telemetry interventions on your model class.** A $+12.5$pp literature result is meaningless if your model never enters the monitored regime — across 960 runs on four 2026 production tiers, the trigger fired zero times.
- **Instrument tool-call engagement as a first-class signal.** When the vendor CLI absorbs the agentic loop, whether the agent uses your provided tools at all is the practitioner-accessible proxy.
- **Distrust trigger conditions assuming syntactic repetition.** Modern frontier models are trained to vary reasoning steps; exact-string repeats no longer occur. Prefer semantic-similarity triggers.

***Abstract*—Production LLM agents across the edge-cloud continuum increasingly run through vendor agent CLIs (Claude, Codex, Gemini) that absorb the agentic loop into the vendor's subprocess. We replicate a peer-reviewed closed-loop intervention [1] — which reported a $+12.5$pp SWE-bench Lite recovery by detecting reasoning loops in real-time telemetry — across four production-tier models from two vendors at $n{=}60$ per arm: 960 instance-runs. The trigger condition (three identical search queries) never fires. Under a passive harness, frontier and mid-tier models one-shot patches without engaging the loop. Under a forced-protocol harness, Opus 4.6 and GPT-5.5 search cleanly but vary every query (max repeat $= 1$ across 960 runs); Sonnet 4.6 and Haiku 4.5 cannot be coerced into the protocol, collapsing patch rates to 3.3–13.3%. The deployment lesson: telemetry interventions tuned on 2024 budget chat models do not transfer to 2026 vendor agent CLIs. We release the harness and traces for reproducibility.**

***Index Terms*—LLM agents, observability, AIOps, OpenTelemetry, closed-loop intervention, deployment experience, edge-cloud continuum, replication study**

The cited prior work [1] is by the same author; this paper deliberately replicates and stress-tests that prior work in a different deployment regime (multi-tier vendor agent CLIs spanning the edge-cloud continuum). All citations are written in third-person form for academic register; the author overlap is disclosed here.

I. Introduction

The edge-cloud continuum increasingly hosts heterogeneous LLM agent deployments. Latency-constrained edge inference at the network perimeter runs budget-tier models like Haiku 4.5 inside vendor CLI containers; high-capability cloud serving runs frontier-tier models like Opus 4.6 or GPT-5.5 in the same CLI runtime but on different physical infrastructure; cost-sensitive batch workloads at regional edge sites run mid-tier models like Sonnet 4.6. The observability stack an AIOps team deploys must remain valid across all three tiers, because the on-call engineer's diagnostic question — did the model hallucinate, did the orchestrator infinite-loop, did a guardrail block the output silently, or did a tool just fail? — is the same question whether the failing agent is running at the edge or in the cloud.

Standard distributed tracing — the OpenTelemetry GenAI semantic conventions [2], for instance — defines operation types for LLM calls, tool execution, and retrieval, but leaves five orchestration phases (planning, reasoning, safety-check monitoring, inter-agent delegation, memory management) as opaque `INTERNAL` spans indistinguishable from any other application logic. Recent peer-reviewed work on agent-specific span taxonomies [1] has proposed nine span kinds to close this observability gap, and reported a *closed-loop intervention* — detecting reasoning loops in real-time via the new `REASONING` span kind and injecting strategy-change prompts — that improved patch recovery on SWE-bench Lite by $+12.5$ percentage points at $n{=}24$.

For practitioners deploying agent observability stacks at the edge-cloud continuum, that published result raises a deployment question: *when I instrument my own agent pipeline with these span kinds, will the same intervention recover the same fraction of my failures at each tier I deploy?* The answer determines whether the instrumentation overhead and

operational complexity of agent-specific telemetry pays for itself, and whether the operations team should plan tier-specific intervention strategies or rely on a single global one.

This paper reports our deployment experience attempting to answer that question. We replicated the published intervention across four production-tier model classes from two vendors — Anthropic's Claude Opus 4.6, Sonnet 4.6, and Haiku 4.5; OpenAI's GPT-5.5 — at n=60 per arm using both a *passive* harness (one that observes whether models engage the tool-call protocol on their own) and a *forced-protocol* harness (one that requires at least three search-tool calls before any patch is accepted). The total experimental matrix comprises 960 instance-runs spanning 16 (model $\times$ harness $\times$ condition) cells.

Our central finding is that the trigger condition — three identical search queries within a single agent run — never fires across the 960 runs. Under the passive harness, all four tiers one-shot patches without engaging the search protocol. Under the forced-protocol harness, Opus 4.6 and GPT-5.5 do engage (averaging 3.2 and 2.6 search calls per run) but vary their queries every time (max repeat $= 1$). Sonnet 4.6 and Haiku 4.5 resist the forced protocol entirely, with patch rates collapsing to 3.3% and 11.7–13.3%.

The deployment implication: telemetry-derived interventions tuned on 2024 budget chat models do not generalize to the vendor CLIs that dominate 2026 production. We argue this is not a flaw in the original intervention but a structural feature of how vendor CLIs absorb the agentic loop into their own runtime, leaving the practitioner-instrumented telemetry layer with nothing to observe.

Contributions:

1) A cross-tier n=60 matched-pair replication of a published closed-loop intervention [1] across four production-tier models from two vendors — 960 instance-runs total.
2) Empirical evidence that the intervention's trigger condition fires zero times across all eight (model $\times$ harness) cells, attributable to two distinct mechanisms: passive bypass (models one-shot without searching) and varied-query engagement (models search but never repeat).
3) A deployment-experience analysis of *vendor agent CLI absorption*: production agent telemetry stacks cannot observe or intervene in tool-use behavior the CLI performs internally.
4) Open-source release of the harness, the 960-trace corpus, and the per-instance JSON dataset for full reproducibility.

II. Background: Agent-Specific Span Kinds and Closed-Loop Intervention

Recent peer-reviewed work [1] introduced an OpenTelemetry-native taxonomy of nine agent-specific span kinds — `AGENT`, `LLM_CALL`, `TOOL_CALL`, `PLANNING`, `REASONING`, `RETRIEVAL`, `GUARD_RAIL`, `DELEGATION`, `MEMORY` — covering execution phases vanilla OpenTelemetry and the GenAI semantic conventions [2] do not represent. Five (`PLANNING`, `REASONING`, `GUARD_RAIL`, `DELEGATION`, `MEMORY`) are novel; the rest overlap with or extend existing conventions. The taxonomy is derived from open coding across seven agent frameworks (Cohen's $\kappa = 0.904$) and validated against the independently developed MAST taxonomy [3] (12 of 14 failure modes covered).

The original work also reported a *closed-loop intervention* that uses these span kinds at runtime: a `REASONING`-span-aware detector watches for the agent calling the same tool with the same arguments three or more times in a row — a pattern interpretable as the agent being stuck in a search loop — and injects a strategy-change prompt encouraging the model to try a different approach. On a 24-instance SWE-bench Lite sample using GPT-4o-mini, the intervention recovered 9 of 24 previously-failed instances (37.5%) versus 6 of 24 (25%) for control, a $+12.5$pp improvement (Fisher's exact two-sided $p = 0.53$). The original authors flagged the small sample size and high p-value as limitations and noted that confirming the effect at $\alpha = 0.05$ with the observed magnitude would require approximately 214 instances per arm.

We focus on whether this intervention transfers to currently-deployed model classes accessed via the vendor agent CLIs that dominate production agent observability in 2026. This is a distinct question from whether the intervention is internally valid on its original conditions — which the original work itself acknowledges as underpowered — and is the question practitioners face when choosing whether to invest in agent-specific telemetry primitives.

III. Method

A. Experimental design

We use the exact same SWE-bench Lite [4] 60-instance sample (drawn from the astropy, django, sympy, and 9 other repositories of the public benchmark) across all eight (model $\times$ harness) cells, with two conditions per cell — control (no intervention) and intervention (with strategy-change prompt injection on the third identical search-query repeat). The intervention trigger and prompt are identical to the original work [1].

B. Two harnesses: passive and forced-protocol

We ran two variants of the harness in order to disambiguate whether the intervention fails because the model never engages the search-tool protocol (passive harness behavior) or because the model engages but in a way that does not exercise the trigger condition (forced harness behavior).

Passive harness (v1). The system prompt offers the agent a search-tool protocol but does not enforce it. The agent is free to one-shot a patch on the first iteration or invoke search tools across multiple iterations as it sees fit. The agent loop runs up to 8 iterations and the intervention prompt fires whenever any single search query has been called three or more times.

Forced-protocol harness (v2). The system prompt explicitly requires at least three `search_code` tool calls before any `<patch>` block will be accepted. If the agent emits a patch before the minimum search count is met, the patch is suppressed and the agent receives a protocol-violation reminder prompt and another iteration. The intervention prompt fires under the same trigger as the passive harness.

The forced-protocol harness was added after we observed that under the passive harness no model engaged the search protocol at all. The intent of the forced harness was to test whether the intervention fires when the protocol is genuinely exercised.

C. Models and CLIs

We tested four production-tier model classes from two vendors:

- **Claude Opus 4.6** (Anthropic frontier) via the Anthropic Claude CLI
- **Claude Sonnet 4.6** (Anthropic mid-tier) via the Anthropic Claude CLI
- **Claude Haiku 4.5** (Anthropic budget) via the Anthropic Claude CLI
- **GPT-5.5** (OpenAI frontier) via the OpenAI Codex CLI

We did not test Anthropic's mid- and budget-tier models on raw API endpoints; all calls go through the vendor agent CLIs that practitioners actually deploy. We also probed Google's Gemini CLI (gemini-3.1-pro-preview, gemini-2.5-pro, gemini-2.5-flash-lite) — all three responded successfully to validation probes, but tier-level n=60 runs are reserved for future work due to the additional engineering required to handle Gemini CLI's substantial output noise.

We attempted to test budget-tier OpenAI models comparable to the GPT-4o-mini used in the original work; the enterprise codex CLI deployment we used exposes only the GPT-5.x family and grok-4, none of which are class-equivalent to GPT-4o-mini. This is itself a deployment-relevance finding: the model class on which the original intervention was developed and reported is no longer accessible via the deployment surface available to practitioners using this CLI in 2026.

D. Outcome metrics and statistics

Per-instance metrics: did the agent produce a parseable patch within the iteration cap (binary); how many iterations were used; how many distinct search queries were issued; the maximum number of times any single search query was repeated; whether the intervention prompt fired. Per-cell statistics: patch rate with Wilson 95% confidence intervals; intervention vs. control 2-by-2 Fisher's exact two-sided p-value.

All experiments use deterministic per-instance random seeds and run end-to-end through the public CLI subprocess interface, ensuring full reproducibility for practitioners who have the same CLI tools installed. The total wall-clock time across all 8 runs was approximately 75 hours of CLI-bound compute, with peak parallelism of three Claude tracks sharing a shared enterprise Anthropic quota.

IV. Results

Table I presents the headline outcomes across all eight (model $\times$ harness) cells. Every number in the table was independently verified against the raw per-instance JSON files via a recomputation script (see Reproducibility, Section VIII).

A. Finding 1: Under passive harness, no model engages the search protocol

The four v1 (passive) cells in Table I show `avg_searches = 0.00` across all four tiers in both conditions. Inspecting per-instance traces confirms this is not a parsing artifact: `avg_iterations` is essentially 1.0 across all four tiers (1.00–1.03), meaning the agent emits a patch on its very first response and exits. The behavior is uniform across the frontier-to-budget range from Opus to Haiku and across both Anthropic and OpenAI vendors.

The intervention prompt cannot fire because the trigger condition (3+ identical search queries) requires the agent to issue at least three search calls; passive-harness agents issue zero. The v1 control-vs-intervention comparison is therefore vacuous: both arms run the identical experiment because the intervention path never activates.

Sonnet 4.6's apparent $+13.3$pp directional effect under v1 ($p = 0.17$) merits comment: this is the only v1 cell where the deltas could plausibly hint at a real effect. Inspection of per-instance traces shows Sonnet's lower baseline patch rate (60% control) is driven not by reasoning loops but by Anthropic Claude CLI gateway timeouts on harder django instances — the model simply does not return within the per-call budget. The intervention prompt never fires on any Sonnet v1 instance, so the $+13.3$pp delta is consistent with sampling noise on differential timeout patterns between the two arms, not with a real intervention effect.

B. Finding 2: Under forced harness, two tiers engage cleanly but vary queries

Opus 4.6 and GPT-5.5 under v2 (forced harness) engage the search protocol substantively: mean search calls are 3.23/3.42 (control/intervention) for Opus and 2.62/2.77 for GPT-5.5, with mean iterations of 4.03/4.03 and 4.47/4.67 respectively. Patch rates remain high — 88.3%/90.0% (Opus) and 83.3%/85.0% (GPT-5.5) — despite the search-first protocol cost. These two cells represent the genuine intervention-testable regime.

And yet the intervention still never fires. The maximum value of `max_query_repeats` observed across all 240 v2-Opus and v2-GPT-5.5 instance-runs is **1**. Every search call within every agent run used a unique query. The intervention's trigger condition (3+ identical search queries) is structurally precluded by the model's behavior: these models, when forced to search, do so by varying their queries rather than repeating.

This is a finer-grained finding than "the agent never gets stuck." The agent may well exhibit semantically-stuck behavior — varying the wording of the same underlying search ten different ways — but the original intervention's trigger is syntactic (exact-string repeat), and the underlying model behavior does not produce exact-string repeats on the modern model tier we tested.

C. Finding 3: Sonnet and Haiku resist the forced protocol

The remaining two v2 cells — Sonnet 4.6 and Haiku 4.5 — present a different failure mode. Patch rates collapse to 3.3% (Sonnet) and 11.7%–13.3% (Haiku), with high

TABLE I

SWE-BENCH LITE n=60 PER-ARM REPLICATION OF THE PUBLISHED CLOSED-LOOP INTERVENTION [1] ACROSS FOUR PRODUCTION-TIER MODELS AND TWO HARNESS VARIANTS. ACROSS ALL 16 (MODEL × HARNESS × CONDITION) CELLS — 960 INSTANCE-RUNS TOTAL — THE INTERVENTION'S TRIGGER CONDITION (3+ IDENTICAL SEARCH QUERIES) *never fires* (`MAX_REPEATS=0` ACROSS ALL V1 RUNS; `MAX_REPEATS=1` ACROSS ALL V2 RUNS). THE PUBLISHED INTERVENTION EFFECT (+12.5PP AT n=24 ON GPT-4O-MINI [1]) DOES NOT REPLICATE AT n=60 ON ANY TESTED TIER.

Harness	Tier	Control	Intervention	Δ	Fisher p	Avg searches
v1 passive	Opus 4.6	51/60 (85.0%)	53/60 (88.3%)	+3.3pp	0.79	0.00 / 0.00
v1 passive	Sonnet 4.6	36/60 (60.0%)	44/60 (73.3%)	+13.3pp	0.17	0.00 / 0.00
v1 passive	Haiku 4.5	55/60 (91.7%)	55/60 (91.7%)	0.0pp	1.00	0.00 / 0.00
v1 passive	GPT-5.5	57/60 (95.0%)	57/60 (95.0%)	0.0pp	1.00	0.00 / 0.00
v2 forced	Opus 4.6	53/60 (88.3%)	54/60 (90.0%)	+1.7pp	1.00	3.23 / 3.42
v2 forced	Sonnet 4.6	2/60 (3.3%)	2/60 (3.3%)	0.0pp	1.00	0.28 / 0.28
v2 forced	Haiku 4.5	8/60 (13.3%)	7/60 (11.7%)	−1.6pp	1.00	1.15 / 1.02
v2 forced	GPT-5.5	50/60 (83.3%)	51/60 (85.0%)	+1.7pp	1.00	2.62 / 2.77

"Avg searches" is mean `search_code` tool calls per run (control / intervention). `max_query_repeats` is 0 across all v1 cells (no searches at all) and 1 across all v2 cells (every search call had a unique query). 95% Wilson confidence intervals on the patch rates are within ±6–13 percentage points across cells; exact bounds in supplementary data.

`patch_suppressions` averages (4.67–5.75 per run): the model attempts to emit a patch every iteration without searching, and our v2 harness rejects each attempt and demands the model search first. The model never satisfies the search-first protocol, the iteration cap is exhausted, and the run terminates with no patch.

Per-instance trace inspection (see Reproducibility, Section VIII) confirms the failure mode: the enterprise Anthropic gateway returns "exit 1" errors after repeated patch-suppression cycles on a substantial fraction of Sonnet/Haiku v2 runs (22–25 errors per 60 runs for Sonnet, 10–12 for Haiku). This is not a model-quality issue; it is a protocol-compliance issue. Sonnet 4.6 and Haiku 4.5 — under the Anthropic Claude CLI deployment we used — cannot be coerced into a user-specified tool-call format by a system prompt alone.

D. Finding 4: Across all eight cells, the intervention's effect size is at or below sampling noise

Across all eight cells, no Δ exceeds 3.3pp in absolute magnitude except v1-Sonnet's +13.3pp (attributable to timeout differential per Finding 1). Every Fisher's exact two-sided p-value is ≥ 0.17, with seven of eight ≥ 0.79 and six of eight at exactly 1.00. None of the deltas approach the published +12.5pp effect with statistical significance, and the underlying mechanism by which the published intervention would deliver any effect (catching reasoning loops) is structurally absent in every cell tested. Given the lowest observed p-value is 0.17 and the headline finding is null, multiple-comparison correction would not change the verdict.

V. THE VENDOR AGENT CLI AS BLACK-BOX AGENT

The dominant pattern is that the agentic loop the original intervention was designed to instrument no longer lives where the practitioner can instrument it. A 2026 practitioner deploying an agent observability stack instruments subprocess invocations of the vendor CLI, not raw LLM API calls. Inside that subprocess, the CLI performs its own tool use, reasoning, and iteration, and reports back a result.

The published work targeted a regime in which the agent loop lives in the practitioner's process and the practitioner's telemetry layer can both observe and inject into it. The 2026 regime — across all four model classes from both vendors we tested — moves the loop inside the vendor's subprocess. The practitioner's stack observes only wrapper-level inputs and outputs. Even when our forced-protocol harness explicitly surfaced the search-tool layer back to the practitioner, the CLI's internal reasoning still produced varied queries that bypass the trigger.

This generalizes beyond the intervention we replicated: any telemetry-derived intervention whose trigger relies on syntactic features of tool use (exact-string repeats, fixed argument matches, specific invocation patterns) is subject to the same risk. As vendor CLIs increase the sophistication of their internal reasoning, the syntactic signal increasingly attenuates.

VI. WHY THIS MATTERS FOR AIOPS AT THE EDGE-CLOUD CONTINUUM

Practitioners deploying LLM agents across the edge-cloud continuum face a model-selection problem: latency-constrained edge inference may run a budget-tier model like Haiku; high-capability cloud serving runs a frontier-tier model like Opus or GPT-5.5; cost-sensitive batch workloads run a mid-tier model like Sonnet. The observability stack must remain valid across all three deployment tiers.

The three practitioner-facing implications appear in the Actionable Insights box at the head of the article. Two warrant brief amplification here. First, the budget-tier 2024 model class on which the original intervention was validated is no longer accessible on at least one major enterprise CLI deployment — the deployment surface itself has moved on. Second, the syntactic-repetition trigger we tested is one of a family; we do not have direct data on related triggers (fixed-argument-match, specific-tool-invocation-pattern), and our finding does not directly refute those. Interventions that look for semantic stuck-state (LLM-judged similarity, embedding-space clustering) may transfer better but are heavier-weight and unvalidated for this deployment regime.

VII. Threats to Validity

Single-benchmark scope. We tested SWE-bench Lite [4] only, matching the original work's benchmark for direct comparison. Generalizability to non-coding tasks — multi-agent orchestration, document QA, τ-bench — is unverified.

Patch-rate as outcome metric. We measure parseable-patch emission within the iteration cap, not SWE-bench official-harness pass — the same metric the original work used, so the comparison is internally valid; absolute patch quality is out of scope.

CLI version dependence. All four CLIs we used were installed via an enterprise gateway infrastructure and may differ in observable behavior from the vendor-published versions a typical external practitioner would use. We document the exact CLI versions in the supplementary materials; the underlying vendor model classes are the same.

Sonnet 4.6 and Haiku 4.5 results may be artifacts of the gateway deployment. The patch-rate collapse (3.3%/11.7%–13.3%) in v2 is partly driven by gateway exit errors after repeated patch-suppression cycles. A non-throttled deployment of the same CLI might exhibit different baseline behavior. The finding that these tiers do not engage the v2 forced protocol is robust (mean searches 0.28–1.15), but the absolute patch rates may not be.

Sample size and baseline-dependent power. At n=60 per arm the experiment is powered (at $\alpha = 0.05$, $\beta = 0.2$) to detect a true $+25$pp effect on cells with intermediate baseline rates, but loses sensitivity sharply where baselines approach the ceiling: Opus v1 (85.0%), GPT-5.5 v1 (95.0%), Haiku v1 (91.7%), Opus v2 (88.3%), GPT-5.5 v2 (83.3%). A $+12.5$pp effect would push GPT-5.5 v1 past 100%. The lower-baseline cells (Sonnet v1 60.0%, Sonnet v2 3.3%, Haiku v2 13.3%) do have headroom but exhibited near-identical patch rates across conditions. Our claim is that the intervention's trigger does not fire — not that we have refuted the original effect's existence at any sample size.

Single-author replication. This paper replicates the present author's own prior work [1] (disclosed in the title footnote). Same-author replication cannot detect shared misunderstandings of the harness's trigger semantics or shared bugs in operationalization. We mitigate by unit-testing the trigger with a synthetic input that fires it by construction (the trigger fires correctly under this input; the 0.0 trigger rate across 960 real runs therefore reflects model behavior, not harness defect). Genuine third-party replication remains future work.

No external instrumentation comparison. We argue vendor CLIs absorb the agentic loop but did not measure what an alternative practitioner-instrumented stack — LangChain+LangSmith, OTel GenAI on raw vendor APIs, or vendor-provided structured logs — would observe. The absorption claim is consistent with what we measured but does not foreclose that some alternative instrumentation could surface portions of the internal loop via stdout/stderr capture or vendor-side telemetry. A side-by-side CLI-wrapped vs. raw-API study would strengthen or refute this claim.

Edge-scope is operational, not experimental. All 960 runs use SWE-bench Lite on cloud-tier compute (Apple M-series). The tier-coverage finding (Section V) directly informs AIOps deployments across the edge-cloud continuum, but we did not measure edge-specific phenomena (constrained bandwidth, intermittent connectivity, edge-side contention). A practitioner running Haiku 4.5 on a network-edge appliance is more likely to see the v2-Haiku patch-rate collapse because the agent loop will not be absorbed by a remote CLI subprocess if the CLI itself is local-only at the edge.

Gemini and grok-4 untested at n=60. Gemini CLI required additional output-parsing work (substantial node-pty and OTel export noise), and grok-4 returned capacity-throttled errors. Generalizing to a third or fourth vendor's CLI is future work.

VIII. Reproducibility

The full pipeline is reproducible from the released artifacts: 960 per-instance trace JSONs, harness source, parser and analysis scripts, and a data-inventory verification script (0 discrepancies across 8 runs $\times$ 9 metrics per cell). We use the public CLI subprocess interface so practitioners with the same CLI tools installed can re-run the harness end-to-end. Per-instance seeds: $\{42, 123, 456, 789, 1024\}$. Hardware: Apple M-series; per-call timeout 600s.

IX. Conclusion

We replicated a published closed-loop intervention for LLM agents [1] across four production-tier model classes from two vendors at n=60 per arm under two harness variants — 960 instance-runs total. The trigger condition never fires in any cell. Under the passive harness, frontier and mid-tier models one-shot patches without engaging the search protocol. Under the forced-protocol harness, two tiers engage but vary every query, structurally precluding the published trigger; the other two cannot be coerced into the protocol by a system prompt alone.

For AIOps practitioners across the edge-cloud continuum: published telemetry interventions tuned on 2024 budget chat models do not generalize to the 2026 vendor agent CLI regime. Invest in tool-call engagement instrumentation as a first-class signal; validate any published intervention's trigger against your specific deployment before adopting; expect interventions relying on syntactic repetition to fail as vendor CLIs absorb more of the agentic loop.

Limitations

The four limitations most likely to affect a practitioner attempting to act on these findings are (in addition to the Threats in Section VII): (L1) we could not test the original GPT-4o-mini class because the enterprise codex CLI exposes only GPT-5.x and grok-4 — the model class the original effect was validated on is not in any production deployment surface we have; (L2) we did not test raw-API access, only the CLI deployment regime that dominates 2026 production; (L3) the forced-protocol harness is itself a deployment-relevant constraint but is not what the original work tested, and the cleanest replication on the original model class is impossible per L1; (L4) the Sonnet/Haiku v2 patch-rate collapse is partly

a gateway artifact, so absolute rates are deployment-dependent though the protocol-resistance finding is robust.

ACKNOWLEDGMENTS

(Camera-ready only.)

REFERENCES

[1] K. C. Balusu, "AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability," in *Proc. 3rd ACM Int'l Conf. on AI-Powered Software (AIware '26)*, Benchmark & Dataset Track, Montreal, QC, Canada, Jul. 6–7, 2026. DOI: 10.1145/3805760.3814931. [Online]. Available: https://2026.aiwareconf.org/track/aiware-2026-benchmark---dataset-track.

[2] OpenTelemetry Authors, "Semantic Conventions for Generative AI Systems," *OpenTelemetry Semantic Conventions*, v1.41.1, released 2026-05-11. [Online; accessed 2026-05-16]. Available: https://github.com/open-telemetry/semantic-conventions/tree/v1.41.1/docs/gen-ai.

[3] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why Do Multi-Agent LLM Systems Fail?" *arXiv preprint arXiv:2503.13657*, 2025.

[4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *Proc. ICLR*, 2024.

[5] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *J. American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[6] R. A. Fisher, *The Design of Experiments*, 1st ed. Edinburgh: Oliver & Boyd, 1935.

AUTHOR BIOGRAPHY

Krishna Chaitanya Balusu is an independent researcher focusing on distributed systems reliability engineering and observability for autonomous AI agents. His research interests include agent observability, deployment-experience studies of telemetry-derived interventions, and reproducibility methodology for systems research. He holds an M.S. in Computer Science from Georgia Institute of Technology and is a Senior Member of IEEE. Contact him at krishnabkc15@gmail.com.